Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

6. Giao dịch phân tán — Saga Pattern . . . 2
6.1. Độ phức tạp của giao dịch phân tán . . . 3
6.1.1. Khi một giao dịch trải rộng trên nhiều dịch vụ . . . 3
6.1.2. Tại sao 2PC không phù hợp? . . . 3
6.2. Saga Pattern — chuỗi Local Transaction + Compensation . . . 6
6.2.1. Định nghĩa . . . 6
6.2.2. KBM Submit Saga . . . 6
6.2.3. Ba loại transaction trong saga . . . 7
6.3. Choreography vs Orchestration . . . 8
6.3.1. Choreography — phi tập trung, event-driven . . . 8
6.3.2. Orchestration — tập trung, điều khiển bằng lệnh . . . 9
6.3.3. Khi nào dùng gì? . . . 10
6.4. Giao dịch bù trừ (Compensating Transaction) & Isolation . . . 11
6.4.1. Thiết kế compensation trong KBM . . . 11
6.4.2. Vấn đề isolation — Anomalies . . . 12
6.4.3. Các biện pháp đối phó (countermeasure) . . . 13
6.4.4. Tổng hợp: anomaly và biện pháp đối phó . . . 15
6.5. Eventual Consistency — chấp nhận và quản lý . . . 16
6.5.1. Định lý CAP và mô hình BASE . . . 16
6.5.2. Consistency window trong KBM . . . 16
6.5.3. Chiến thuật cho giao diện và nghiệp vụ . . . 17
6.6. Case study: KBM . . . 17
6.6.1. Hiện trạng: Implicit Saga (Choreography) . . . 17
6.6.2. Phân tích các vấn đề và lộ trình cải tiến . . . 19
6.6.3. Workflow học vụ cũng là saga . . . 21
6.7. Tổng kết . . . 22
6.8. Đọc thêm . . . 22
Tài liệu tham khảo . . . 23

CHƯƠNG 6

Giao dịch phân tán — Saga Pattern

> "A saga is a sequence of local transactions. Each local transaction updates data within a single service using the familiar ACID transaction frameworks and libraries..."
> — Chris Richardson, *Microservices Patterns* [2a, Ch.4]

MỤC TIÊU HỌC TẬP

- Hiểu tại sao giao dịch phân tán (2PC) không phù hợp với microservices
- Nắm vững Saga pattern: định nghĩa, cấu trúc, và giao dịch bù trừ (compensating transaction)
- So sánh hai cơ chế điều phối: Choreography vs Orchestration
- Áp dụng các biện pháp đối phó (countermeasure) để xử lý việc thiếu isolation giữa các saga
- Hiểu eventual consistency và cách quản lý từ góc nhìn business
- Phân tích luồng nộp bài trong KBM — implicit saga và lộ trình cải tiến

Chuyển sang microservices mang lại khả năng mở rộng độc lập và triển khai linh hoạt, nhưng đồng thời phá vỡ nền tảng nhất quán dữ liệu truyền thống: transaction không còn gói gọn trong một database. Chương này giải quyết bài toán giao dịch phân tán bằng Saga pattern — từ nguyên lý đến cách KBM áp dụng cho luồng nộp bài.

Ôn lại kiến thức nền

Chương này dựa nhiều vào một số khái niệm cơ sở dữ liệu. Nếu chưa chắc, hãy nắm nhanh bốn ý sau (ví dụ trừ tiền trong thẻ sinh viên (ví e-learning) để nạp quota in ấn ở thư viện):

- **Transaction**: một nhóm thao tác "tất cả hoặc không có gì". Nạp quota phải *trừ* tiền thẻ sinh viên *và cộng* quota in ấn cùng lúc; không thể trừ tiền xong rồi dừng giữa chừng.
- **ACID**: bốn đảm bảo của transaction trong một database (Atomicity, Consistency, Isolation, Durability). Chữ **I — Isolation** nghĩa là các giao dịch chạy song song không gây xung đột hay can thiệp lẫn nhau.
- **Anomaly (bất thường)**: khi thiếu isolation, hai thao tác đồng thời có thể cho kết quả sai: ví dụ hai luồng cùng đọc số dư thẻ 100k, cùng trừ 80k → số dư cuối sai (*lost update*).
- **Idempotency**: một thao tác chạy lại nhiều lần vẫn cho cùng kết quả như chạy một lần. "Ghi số dư = 20k" là idempotent; "cộng thêm 20k" thì *không* (chạy 2 lần thành 40k).

Vấn đề trung tâm của chương: khi một "transaction" trải trên **nhiều service với database riêng**, chúng ta *không còn* ACID xuyên suốt — và Saga là cách sống chung với điều đó.

6.1. Độ phức tạp của giao dịch phân tán

6.1.1. Khi một giao dịch trải rộng trên nhiều dịch vụ

Nếu toàn bộ luồng nộp bài nằm trong một monolith, nó gói gọn trong một database transaction (minh họa):

Listing 6.1: *Transaction trong monolith — ACID đảm bảo tất cả cùng thành công hoặc cùng rollback*

```
BEGIN TRANSACTION;
  INSERT INTO submissions (id, user_id, sql_content) VALUES (...);
  INSERT INTO judge_queue (submission_id, status) VALUES (..., 'PENDING');
  UPDATE user_stats SET total_submissions = total_submissions + 1
    WHERE user_id = ...;
COMMIT;
-- Tất cả thành công hoặc tất cả rollback — ACID đảm bảo
```

Khi KBM chuyển sang microservices, dữ liệu không còn nằm chung một chỗ: Core Service lưu bài nộp trong database của mình, Judge Service ghi việc chấm vào database riêng (`judge_db`) và chạy bài trong sandbox, Notification Service đẩy thông báo. Không có "shared transaction" giữa chúng — Core không biết Judge đã chấm xong hay chưa nếu không có thông điệp báo về.

6.1.2. Tại sao 2PC không phù hợp?

Để hiểu lý do Saga Pattern ra đời, trước tiên chúng ta cần nhìn lại cách các hệ thống truyền thống xử lý giao dịch phân tán. Từ nhiều thập kỷ qua, **Two-Phase Commit (2PC)** là cơ chế tiêu chuẩn để giải quyết bài toán đồng bộ hóa dữ liệu trên nhiều nút mạng. Ý tưởng cốt lõi của 2PC được chia thành hai giai đoạn dưới sự dẫn dắt của một thành phần trung tâm gọi là Coordinator.

Trong giai đoạn đầu (Prepare Phase), Coordinator sẽ gửi yêu cầu tới tất cả các cơ sở dữ liệu thành phần (participant), hỏi xem chúng đã sẵn sàng để ghi dữ liệu hay chưa. Chỉ khi tất cả các bên đều phản hồi "Sẵn sàng", đã ghi bền vững phần chuẩn bị và giữ khóa (lock) trên dữ liệu liên quan, Coordinator mới ghi lại quyết định commit rồi chuyển sang giai đoạn thứ hai (Commit Phase), gửi quyết định đó tới từng bên. Sự phối hợp này bảo đảm mọi bên cùng commit hoặc cùng abort; các thuộc tính còn lại của ACID, như isolation, vẫn do cơ chế giao dịch của từng cơ sở dữ liệu quyết định.

Dưới góc nhìn lý thuyết theo Kleppmann [7, Ch.9], 2PC dường như là giải pháp an toàn để bảo toàn sự nhất quán dữ liệu ở mức độ cao nhất. Tuy nhiên, khi áp dụng cho hệ thống phân mảnh cao và yêu cầu xử lý lớn, cơ chế này bộc lộ những hạn chế lớn về hiệu năng và khả năng chịu lỗi.



Hình 6.1: *Two-Phase Commit — Coordinator điều phối prepare và commit*

Richardson trong [2a, Ch.4] liệt kê các lý do chính khiến 2PC (giao dịch XA — một chuẩn kỹ thuật phổ biến cài đặt 2PC) không phù hợp cho microservices, đặc biệt khi áp dụng vào bối cảnh hệ thống thực tế như KBM:

- **Synchronous blocking**: Bộ điều phối (Coordinator) bị chặn (block) để chờ phản hồi từ tất cả các thành phần tham gia (participant); đồng thời, mỗi thành phần tham gia phải duy trì khóa giao dịch trên tài nguyên của nó cho đến khi nhận lệnh commit/abort. Trong KBM, Judge Service xử lý một bài nộp SQL có thể mất từ 5–30 giây. Điều này đồng nghĩa với việc thành phần tham gia sẽ giữ row lock và bộ điều phối bị chặn chờ trong toàn bộ khoảng thời gian đó, khiến luồng xử lý của Core Service bị giữ suốt thời gian đó, và thread pool sẽ cạn khi nhiều bài nộp cùng chờ.
- **Single point of failure**: Nếu bộ điều phối trung tâm gặp sự cố ngừng hoạt động (crash), tất cả các thành phần tham gia sẽ bị đình trệ ở trạng thái chờ. Với KBM, nếu bộ điều phối gặp

sự cố ngắt kết nối ngay trong lúc Judge đang chạy, toàn bộ tiến trình nộp bài sẽ bị đình trệ vô thời hạn.

- **Reduced availability**: Giao thức yêu cầu tất cả các thành phần tham gia giao dịch phải trực tuyến cùng lúc — độ khả dụng tổng thể xấp xỉ tích độ khả dụng của từng mắt xích, nếu coi lỗi của chúng là độc lập. Chẳng hạn, nếu một giao dịch nộp bài cập nhật điểm dùng 2PC trải qua `kbm-core`, `kbm-academic` và `kbm-notification`, chỉ cần `kbm-notification` tạm ngưng bảo trì là toàn bộ luồng nộp bài cũng bị từ chối theo.
- **NoSQL incompatible**: Nhiều công nghệ lưu trữ hiện đại không hỗ trợ giao dịch XA. Ví dụ, Kafka — hệ thống truyền tin cốt lõi (messaging backbone) của KBM — hoàn toàn không hỗ trợ giao thức XA, khiến việc áp dụng 2PC là bất khả thi.
- **Performance bottleneck**: Việc giữ lock trong thời gian dài dẫn đến mức độ contention (cạnh tranh tài nguyên) rất cao. Trong chế độ thi của KBM — 200 sinh viên cùng ca thi, cao điểm cuối giờ hàng trăm lượt nộp mỗi phút theo kịch bản chất lượng ở §1.5 — cơ chế khóa này sẽ tạo ra một nút thắt cổ chai (bottleneck) làm suy giảm hiệu năng toàn hệ thống.

Kleppmann trong [7, Ch.9] bổ sung: 2PC là "blocking protocol" — nếu bộ điều phối gặp sự cố sau giai đoạn một, các participant phải chờ vô hạn. Trên production, trong chế độ thi, đây là một rủi ro không thể chấp nhận được.

2PC mâu thuẫn với Microservices

Two-Phase Commit yêu cầu tất cả participant blocking và đồng bộ — điều này trực tiếp vi phạm mục tiêu triển khai độc lập (independent deployment) và tính sẵn sàng cao (high availability) của hệ thống microservices. Khi coordinator gặp sự cố, các giao dịch đang dở bị treo và giữ khóa cho đến khi coordinator phục hồi. Có thể giảm rủi ro này bằng cách nhân bản coordinator, nhưng đổi lại là thêm độ trễ và độ phức tạp, trái với mục tiêu các service tự chủ.

Sự khác biệt này đòi hỏi thay đổi tư duy thiết kế khi chuyển sang Microservices. Trong kiến trúc monolith, các cơ chế khóa đồng bộ (synchronous locking) đảm bảo tính toàn vẹn một cách tự nhiên. Khi một giao dịch bắt đầu, cơ sở dữ liệu sẽ giữ chặt các bản ghi liên quan, ngăn cản bất kỳ sự can thiệp nào từ bên ngoài cho đến khi giao dịch hoàn tất.

Tuy nhiên, trên hệ thống phân tán như KBM, nơi mạng không đáng tin cậy, việc khóa tài nguyên bằng 2PC trở thành điểm nghẽn (bottleneck). Khi quy mô tăng, xác suất tắc nghẽn cục bộ hoặc lỗi dây chuyền càng cao. Do đó, thiết kế hệ thống phải chấp nhận tính nhất quán cuối cùng (eventual consistency) thay vì nhất quán tức thời (strong consistency).

Hệ thống phân tán hiện đại ưu tiên khả năng phục hồi (resilience) hơn là khóa tài nguyên: lỗi không bị ngăn hoàn toàn mà được xử lý qua các giao dịch bù trừ (compensating transaction). Đó là nền tảng của Saga Pattern.

Compensation thay vì Locking

Nghĩ về quy trình nộp bài thực tế: sinh viên nộp SQL → hệ thống nhận bài → Judge chấm → trả kết quả. Nếu Judge gặp lỗi, chúng ta không "hoàn tác" việc nhận bài — chúng ta *cập nhật trạng thái* thành ERROR và thông báo sinh viên thử lại. Đây chính là tư duy compensation — hành động nghiệp vụ ngược thay vì database rollback. Richardson trong [2a, Ch.4] đề cập đến quan sát của Gregor Hohpe: "Not even Starbucks uses two-phase commit" [xem Đọc thêm]. Khi khách gọi cà

phê, thu ngân nhận tiền rồi đưa biên lai, sau đó khách ra góc chờ Barista pha cà phê (bất đồng bộ). Nếu Starbucks dùng 2PC, thu ngân sẽ yêu cầu khách hàng chờ đợi, đồng thời phong tỏa trạng thái của cả thu ngân và nhân viên pha chế cho đến khi ly cà phê được hoàn thành rồi mới thực hiện thanh toán. Saga hoạt động theo cùng nguyên tắc này.

6.2. Saga Pattern — chuỗi Local Transaction + Compensation

Saga hoạt động giống như quy trình xét duyệt đồ án qua nhiều phòng ban. Mỗi bước hoàn tất sẽ tự động kích hoạt bước tiếp theo. Nếu xảy ra lỗi giữa chừng, hệ thống sẽ chạy các nghiệp vụ bù trừ để hoàn tác lại những thay đổi trước đó.

6.2.1. Định nghĩa

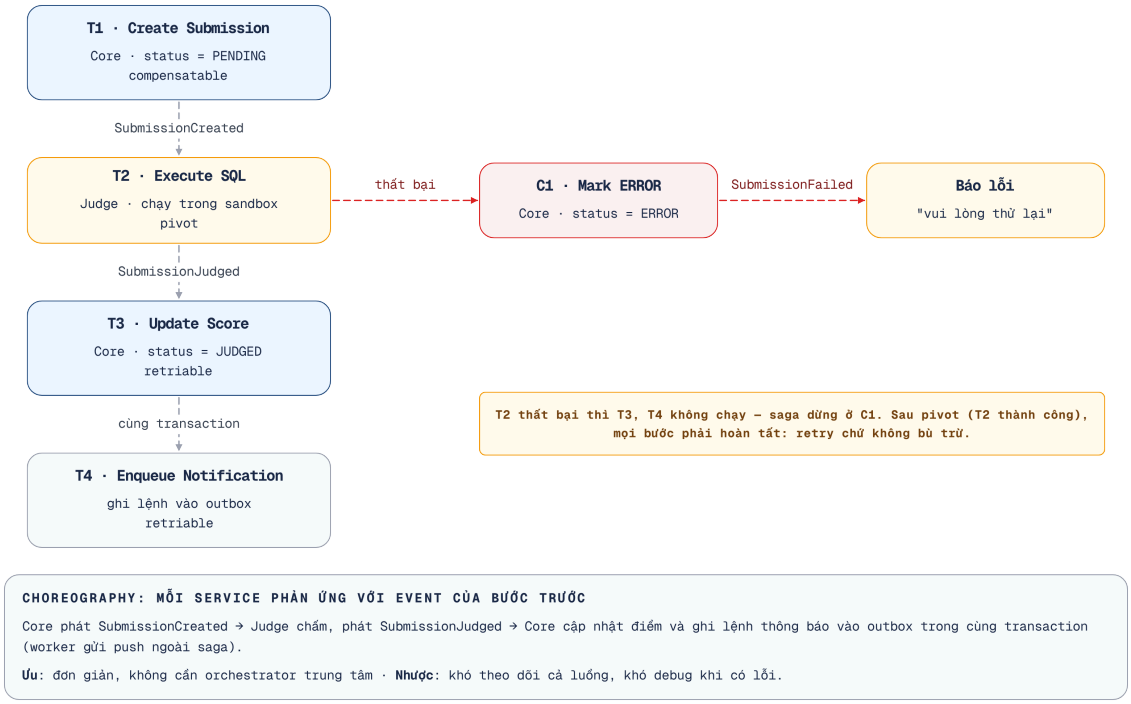
Saga là chuỗi **giao dịch cục bộ (local transaction)**, mỗi transaction cập nhật database của một service và phát đi các sự kiện hoặc thông điệp để kích hoạt giao dịch tiếp theo. Nếu một transaction thất bại, saga thực hiện **giao dịch bù trừ (compensating transaction)** để hoàn tác các thay đổi trước đó [2a, Ch.4].

6.2.2. KBM Submit Saga

Để làm rõ cách thức hoạt động của Saga, hãy xét luồng nghiệp vụ cốt lõi của KBM: quy trình nộp và chấm bài tập SQL. Khi sinh viên nộp bài, thay vì tạo ra một giao dịch phân tán khóa toàn bộ các bảng dữ liệu liên quan, hệ thống phân rã tiến trình này thành một chuỗi các giao dịch cục bộ độc lập (local transaction).

Mỗi dịch vụ trong hệ thống sẽ tự chịu trách nhiệm cập nhật kho dữ liệu riêng biệt của dịch vụ đó, sau đó sử dụng các sự kiện (event) thông qua hệ thống message broker như Kafka để truyền tín hiệu báo hiệu cho dịch vụ tiếp theo trong chuỗi. Tuy nhiên, sự phân tách này để lại một vấn đề bắt buộc phải xử lý: khi một bước giữa chuỗi thất bại, những bước đã hoàn tất trước đó vẫn nằm lại trong cơ sở dữ liệu của từng dịch vụ.

Lúc này, cơ chế giao dịch bù trừ (compensating transaction) được kích hoạt. Nếu Core Service đã ghi nhận bài nộp nhưng Judge Service lại gặp lỗi khi khởi tạo sandbox, hệ thống không thể rollback database của Core Service thông qua ACID. Thay vào đó, một sự kiện bù trừ được phát đi, yêu cầu Core Service cập nhật trạng thái bài nộp thành “Lỗi” và thông báo cho sinh viên. Chuỗi các giao dịch cục bộ và giao dịch bù trừ này chính là một Saga.



Hình 6.2: KBM Submit Saga — chuỗi local transaction và compensating transaction

Richardson [2a, Ch.4] minh họa một saga điển hình qua luồng đặt hàng thương mại điện tử, trong đó bước cấp phép thanh toán thẻ tín dụng là một giao dịch bản lề (pivot transaction) quyết định kết quả của toàn chuỗi. Đối chiếu với KBM, luồng nộp bài (Submit Saga) có ít dịch vụ tham gia hơn (Core Service, Judge Service) nhưng bước chấm chạy lâu nên việc quản lý trạng thái vẫn cần cẩn thận.

Điểm khác biệt lớn nhất nằm ở bản chất của quá trình thực thi chấm bài (Judge execution) — một tác vụ mang tính chất **long-running** (chạy nền trong thời gian dài). Trong khi việc cấp phép thanh toán thẻ thường xong trong khoảng một giây, một tiến trình chạy sandbox để biên dịch, khởi tạo dữ liệu mẫu và chạy thử các câu truy vấn SQL của sinh viên có thể kéo dài từ vài giây cho đến vài chục giây.

Thời gian thực thi dài đồng nghĩa với một asynchronous window rộng hơn hẳn — tức là khoảng thời gian mà các thao tác đồng thời hoặc lỗi mạng có thể chen ngang cũng dài theo. Hệ quả thiết kế là: không thể đối xử với mọi bước trong saga như nhau, mà phải phân loại chúng theo mức độ rủi ro, khả năng hoàn tác và tính bắt buộc phải hoàn thành.

6.2.3. Ba loại transaction trong saga

Richardson phân loại mỗi bước trong một chuỗi saga thành ba loại giao dịch riêng biệt, giúp định hình rõ ràng ranh giới và cách xử lý lỗi tại từng thời điểm. Áp dụng cơ sở lý thuyết này vào KBM Submit Saga, chúng ta có:

- **Compensatable Transaction (Giao dịch có thể bù trừ)**: Đây là những bước nằm ở phần đầu của saga, có thể bị hoàn tác (undo) một cách an toàn thông qua các giao dịch bù trừ (compensating transaction). Ở ví dụ KBM, đó là T1 (Create Submission). Nếu có lỗi xảy ra ở bước sau, chúng ta chạy C1 (Mark ERROR) để bù trừ lại.

- **Pivot Transaction (Giao dịch bản lề)**: Đây là "điểm không thể quay đầu", tương tự như thời điểm khoản thanh toán thẻ tín dụng của khách hàng được cấp phép. Sau bước này, toàn bộ saga bắt buộc phải được hoàn thành và không được phép bù trừ nữa. Bản thân pivot không có bù trừ. Trong KBM, T2 (Execute SQL) chính là bước pivot: thao tác chạy mã SQL trong sandbox sẽ thành công hoặc thất bại, quyết định số phận của toàn bộ luồng.
- **Retriable Transaction (Giao dịch có thể thử lại)**: Những bước đi theo sau điểm pivot. Chúng mang đặc tính là *bắt buộc* phải thành công và không cần giao dịch bù trừ. Nếu gặp lỗi kết nối hay timeout, hệ thống sẽ thực hiện thử lại (retry) cho đến khi hoàn tất. KBM có hai bước retriable là T3 (Update Score) và T4 (Notify Student).

Lưu ý thực tế về T4: "retry đến khi thành công" giả định bước đó chắc chắn có thể thành công (như ghi điểm vào database nội bộ). Vì việc gửi email/push thực tế có thể thất bại vĩnh viễn (địa chỉ hỏng), bước retriable đúng nghĩa của T4 nên là *ghi bền vững lệnh thông báo* — enqueue vào outbox (bảng "hộp thư đi" ghi trong cùng transaction nội bộ nên luôn có thể thành công; chi tiết pattern này ở Chương 10). Còn việc phát thông báo thực tế chạy *ngoài* saga theo kiểu best-effort với retry giới hạn, backoff và Dead Letter Queue — một thông báo hỏng địa chỉ không được phép giữ saga "treo" vô hạn hay tạo vòng lặp poison pill.

Việc cấu trúc đúng ba phần này là yêu cầu bắt buộc khi thiết kế Saga. Trình tự thực thi phải luôn là: bắt đầu với các bước **Compensatable**, vượt qua điểm **Pivot**, và kết thúc bằng các bước **Retriable**. Nếu đặt một bước có khả năng thất bại (nhưng không thể bù trừ) vào cuối chuỗi, hệ thống sẽ rơi vào trạng thái bất nhất. Trong luồng nộp bài của KBM, T2 (Execute SQL) là điểm Pivot. Một khi T2 thành công, saga chắc chắn phải đi đến đích. Các bước tiếp theo như T3 (Update Score) và T4 (Notify Student) là Retriable. Nếu các bước này gặp lỗi mạng hay timeout, hệ thống sẽ kiên trì thử lại cho đến khi thành công (với backoff; phần phát thông báo thực tế được tách ra ngoài saga như đã lưu ý ở trên), và tuyệt đối không kích hoạt giao dịch bù trừ để hủy bỏ kết quả T2.

Hiểu đúng nghĩa 'Pivot'

T2 là pivot theo nghĩa: *nếu thành công*, saga cam kết tiến tới và không quay đầu. Cần phân biệt rõ trường hợp T2 *thất bại*: đó không phải là "compensation cho pivot" (pivot không có compensation), mà là tín hiệu để rollback các bước **compensatable** đứng trước nó — ở đây là chạy C1 để đánh dấu submission ERROR. Nói cách khác, một bước pivot thất bại sẽ kích hoạt compensation của T1, chứ bản thân pivot không bị undo.

6.3. Choreography vs Orchestration

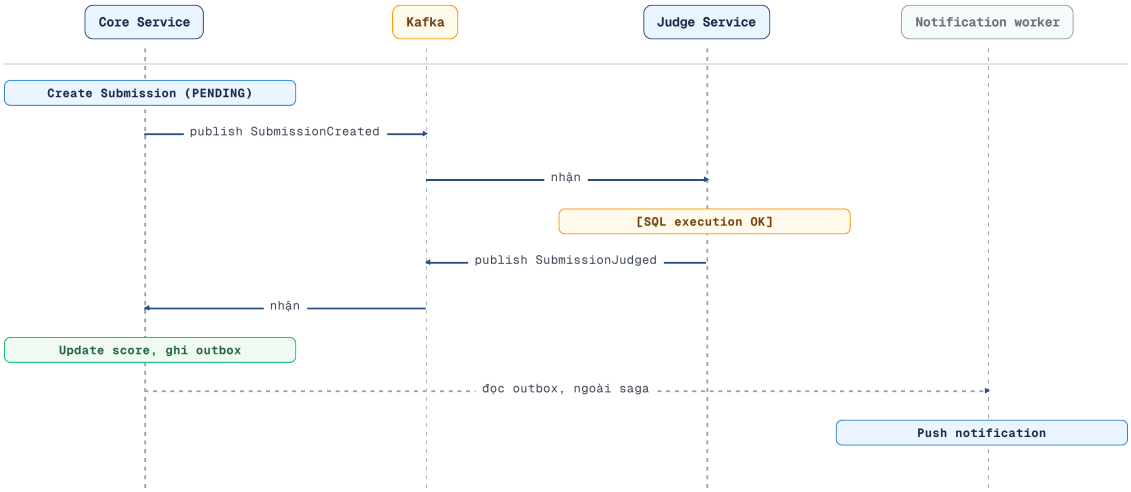
Để các dịch vụ phối hợp nhịp nhàng, hệ thống cần một nhạc trưởng hoặc các quy tắc tự quản. Việc chọn mô hình điều phối giống như quyết định cho sinh viên tự do đăng ký tín chỉ hay bắt buộc phải thông qua sự duyệt xét của cố vấn.

6.3.1. Choreography — phi tập trung, event-driven

Khi đã định hình rõ rệt các mảnh ghép tạo nên một Saga, vấn đề tiếp theo là làm thế nào để kết nối chúng. Có hai mô hình chính để giải quyết bài toán điều phối này. Mô hình đầu tiên là **Choreography**, trong đó hệ thống hoạt động hoàn toàn phi tập trung và **không có coordinator** (bộ điều phối trung tâm).

Hình ảnh so sánh gần gũi nhất là một nhóm sinh viên tổ chức sự kiện: Mỗi nhóm sinh viên trong ban tổ chức tự biết nhiệm vụ của mình; nhóm hậu cần hoàn tất công việc sẽ tự thông báo để nhóm truyền thông bắt đầu, không cần một trưởng ban (coordinator) điều phối từng bước.

Đặt vào ngữ cảnh kỹ thuật, mỗi service trong hệ thống sẽ âm thầm lắng nghe các sự kiện (event) được phát ra trên một kênh truyền thông điệp chung (như Kafka), và tự động đưa ra quyết định thực thi các hành động tiếp nối dựa trên logic nội tại của chính nó [5, §4.2]. Luồng nộp bài của KBM hiện cũng chạy theo kiểu này.



Hình 6.3: *Choreography — các service tự phối hợp qua event*

Mô hình Choreography rất **đơn giản** trong giai đoạn đầu vì không yêu cầu một thành phần điều phối tập trung. Các dịch vụ hoạt động hoàn toàn **loosely coupled** (ràng buộc lỏng lẻo); chúng không cần biết đến sự tồn tại của nhau mà chỉ phản ứng dựa trên các sự kiện (event).

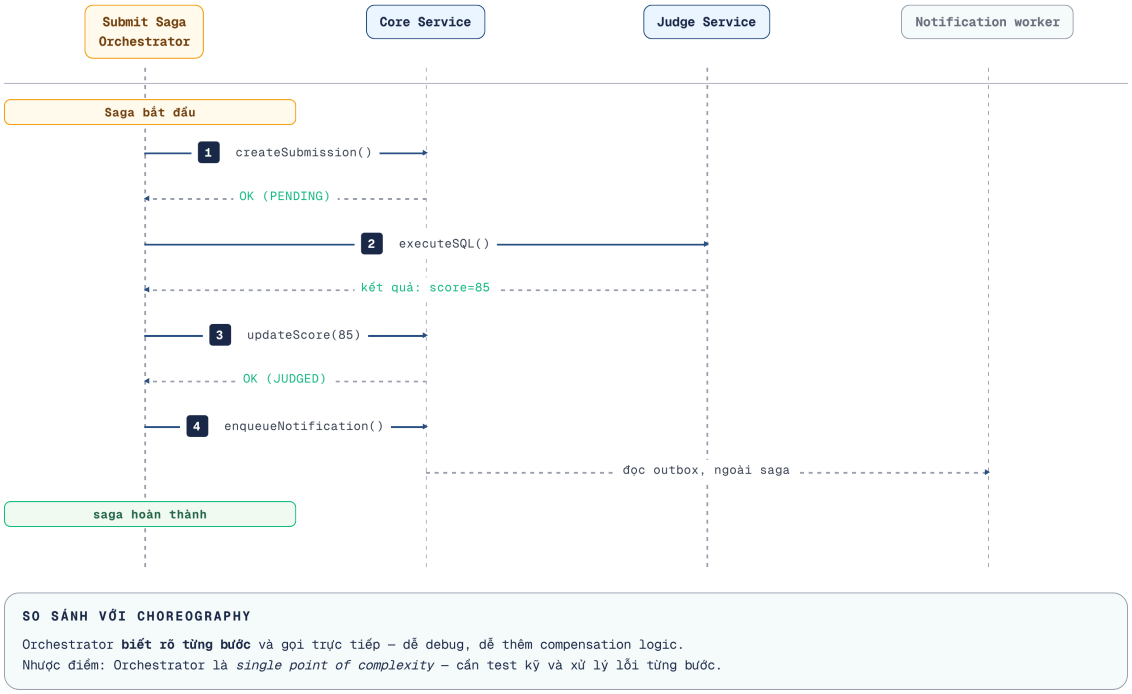
Sự độc lập này giúp quá trình **thêm mới thành phần tham gia** trở nên dễ dàng: một dịch vụ mới chỉ cần đăng ký (subscribe) vào luồng sự kiện tương ứng mà không làm ảnh hưởng dịch vụ cũ. Tuy nhiên, độ phức tạp sẽ tăng cao khi hệ thống mở rộng.

Luồng xử lý tổng thể dần trở nên **khó theo dõi** do logic bị phân tán. Nếu thiết kế không tốt, hệ thống dễ rơi vào **cyclic dependencies** (phụ thuộc vòng) — service A đăng ký sự kiện của B, rồi B lại đăng ký sự kiện của A. Hơn nữa, việc **kiểm thử (testing)** toàn bộ luồng đòi hỏi phải chạy đồng thời tất cả dịch vụ tham gia.

6.3.2. Orchestration — tập trung, điều khiển bằng lệnh

Khác với Choreography, mô hình **Orchestration** (Điều phối) sử dụng một thành phần trung tâm gọi là **Saga Orchestrator**. Orchestrator hoạt động như một lớp trưởng hoặc trưởng ban tổ chức, nắm quyền kiểm soát và điều phối toàn bộ luồng thực thi của Saga bằng cách trực tiếp giao việc cho từng thành viên và chờ báo cáo.

Orchestrator không chờ đợi các sự kiện xảy ra một cách thụ động, mà nó chủ động phát đi các mệnh lệnh (command) rõ ràng đến từng dịch vụ con và cẩn thận chờ đợi phản hồi (reply) trả về để quyết định bước đi tiếp theo [2a, Ch.4]. Nếu áp dụng mô hình orchestrator tập trung vào luồng nộp bài (Submit Saga) của KBM, kiến trúc sẽ thay đổi rõ rệt.



Hình 6.4: Orchestration — saga orchestrator điều phối toàn bộ luồng xử lý

Ưu điểm lớn nhất của Orchestration là tính **dễ hiểu** và sự rõ ràng của luồng xử lý. Toàn bộ logic điều hướng được tập trung tại một nơi, giúp kỹ sư dễ dàng theo dõi trạng thái saga. Kiến trúc này loại bỏ rủi ro phụ thuộc vòng (**không có cyclic dependencies**), do biểu đồ gọi dịch vụ luôn đi theo một chiều từ orchestrator xuống các dịch vụ con.

Về mặt đảm bảo chất lượng, khâu **kiểm thử (testing) cũng trở nên đơn giản hơn**. Các lập trình viên có thể tạo ra các bản giả lập (mock) cho orchestrator để kiểm thử độc lập tính đúng đắn của từng dịch vụ con mà không cần phải dựng cả một hệ sinh thái cồng kềnh. Đổi lại những lợi ích quản trị, mô hình này phải chấp nhận **centralization risk**: orchestrator có thể trở thành single point of failure nếu chỉ chạy một instance và không lưu trạng thái saga bền vững.

Ngoài ra, nếu không phân chia trách nhiệm rõ ràng, hệ thống dễ rơi vào anti-pattern **smart orchestrator**: business logic bị đẩy hết vào orchestrator, biến nó thành một monolith, còn các dịch vụ con thoái hóa thành **anemic service** — chỉ còn lại thao tác đọc/ghi cơ bản, không nắm giữ nghiệp vụ nào.

6.3.3. Khi nào dùng gì?

Rocha trong [5, §4.4] đề xuất một góc nhìn đa chiều để kết hợp cả hai cơ chế, và đây được xem là cách tiếp cận thực tế nhất:

- **Saga đơn giản (ít bước, tuần tự, không phân nhánh)**: Nên ưu tiên **Choreography**. Ví dụ thực tế chính là luồng nộp bài (Submit flow) hiện tại của KBM (4 bước T1–T4, 2 dịch vụ, không phân nhánh): áp dụng choreography là vừa đủ, không gây phức tạp hóa hệ thống.
- **Saga phức tạp (nhiều hơn 4 bước hoặc có phân nhánh)**: **Orchestration** là giải pháp phù hợp/cần thiết. Đối với KBM, nếu trong tương lai luồng nộp bài mở rộng, cần tích hợp thêm các dịch vụ như kiểm tra đạo văn (plagiarism check) hoặc chấm điểm dựa trên rubric, số lượng nhánh rẽ tăng lên sẽ đòi hỏi một orchestrator để kiểm soát mạch lạc.

- **Giao tiếp giữa các Bounded Context**: Nên sử dụng **Choreography** để giữ sự độc lập và lỏng lẻo (loose coupling) giữa các ranh giới ngữ cảnh. Giao tiếp giữa Core Service và Judge Service trong KBM là minh chứng điển hình cho việc kết nối hai domain khác biệt.
- **Nhóm phát triển chưa quen với EDA (Event-Driven Architecture)**: **Orchestration** thường dễ nắm bắt và dễ gỡ lỗi (debug) hơn, phù hợp với một đội ngũ đang trong quá trình mở rộng hoặc chưa có nhiều kinh nghiệm về luồng sự kiện phi tập trung.

Điểm mấu chốt không phải chọn phe, mà là nhận ra đúng thời điểm luồng nghiệp vụ đã đủ phức tạp để cần một bộ điều phối tường minh.

Với luồng nộp bài hiện tại (4 bước, 2 dịch vụ, không phân nhánh), **choreography là đủ** — việc thêm bộ điều phối sẽ làm thiết kế phức tạp hơn mức cần thiết. Tuy nhiên, nếu tương lai luồng nghiệp vụ mở rộng (thêm plagiarism detection service, thêm grading rubric service, thêm analytics service), orchestration trở nên cần thiết. Đây là quyết định kiến trúc mở — ghi nhận để xem xét lại khi yêu cầu bài toán thay đổi.

6.4. Giao dịch bù trừ (Compensating Transaction) & Isolation

Bù trừ dữ liệu chính là các nghiệp vụ thực tế. Khi phòng đào tạo phát hiện sai sót lịch học, họ không thể quay ngược thời gian để xóa đi, mà phải phát hành thông báo đính chính và chủ động sắp xếp lại một lịch học mới cho sinh viên.

6.4.1. Thiết kế compensation trong KBM

Compensation không phải là một lệnh "undo" mù quáng ở mức cơ sở dữ liệu. Về bản chất, nó là một **hành động nghiệp vụ ngược** có ý nghĩa thực tiễn [2a, Ch.4]. Việc thiết kế các hành động này phải gắn liền với logic của từng dịch vụ. Trong ngữ cảnh KBM, cơ chế này được phân rã như sau:

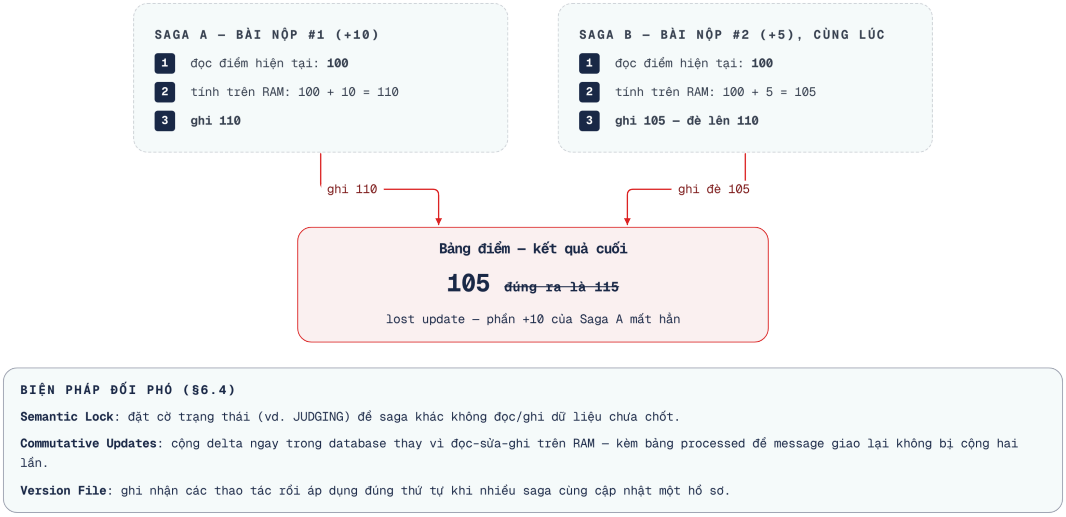
- **T1: Create Submission (Trạng thái PENDING)**: Hành động bù trừ tương ứng là C1 (Mark ERROR). Thay vì xóa bỏ (DELETE) bản ghi submission trong cơ sở dữ liệu một cách trực tiếp, hệ thống chỉ cập nhật trạng thái của nó thành ERROR. Việc này giữ lại lịch sử để hỗ trợ đối soát sau này.
- **T2: Execute SQL**: Đây là điểm Pivot nên không có hành động compensation. Nếu tiến trình gặp lỗi, Judge Service sẽ tự động dọn dẹp (cleanup) môi trường sandbox tạm thời của nó.
- **T3: Update Score và T4: Enqueue Notification Command**: Đây là các bước Retriable, đồng nghĩa với việc chúng không bao giờ kích hoạt compensation hay rollback. Cả hai đều là thao tác *nội bộ* (ghi điểm vào database, ghi lệnh thông báo vào outbox) nên luôn có thể thử lại đến khi thành công. Việc gửi email/push thực tế nằm *ngoài* saga: notification worker đọc outbox và gửi theo kiểu best-effort với retry giới hạn, backoff và Dead Letter Queue.

Nguyên tắc cốt lõi: Mỗi compensation phải được thiết kế sao cho nó **idempotent** — nghĩa là có thể được gọi hàng chục lần nhưng vẫn chỉ cho ra duy nhất một kết quả như lần gọi đầu tiên. Với KBM, `markError(submissionId)` cần được viết sao cho gọi lặp do lỗi mạng vẫn chỉ ghi ERROR một lần, không sinh side effect (tác dụng phụ) nào. Lưu ý quan trọng: sau khi vượt qua Pivot transaction (T2), tất cả Retriable transaction (T3, T4) **bắt buộc** phải được thực thi đến cùng — điều này khả thi vì chúng đều là thao tác nội bộ. Chúng không được phép rollback hay kích hoạt compensate, vì lúc này Saga đã được xem là hoàn tất ở cấp độ nghiệp vụ. Ngược lại, một bản ghi nằm trong DLQ nghĩa là việc gửi thông báo *chưa* thành công — DLQ là nơi chờ đối soát, không phải bằng chứng bước đó đã hoàn tất; đó cũng chính là lý do việc gửi thực tế được tách ra khỏi saga.

6.4.2. Vấn đề isolation — Anomalies

Do Saga là chuỗi các giao dịch cục bộ, kết quả trung gian chưa hoàn tất của một saga có thể bị đọc bởi các saga chạy đồng thời khác. Richardson [2a, Ch.4] gọi đây là hiện tượng **lack of isolation**. Sự thiếu vắng Isolation trực tiếp gây ra ba loại bất thường (anomaly) về dữ liệu, cần xử lý bằng **các biện pháp đối phó (countermeasure)** thủ công:

1. **Lost Update (Mất dữ liệu cập nhật)** — Hiện tượng này xảy ra khi Saga A vô tình ghi đè lên những thay đổi mà Saga B vừa mới lưu xuống thành công, trong khi Saga A hoàn toàn không có khái niệm gì về việc bản ghi đó đã bị thay đổi kể từ lúc nó đọc dữ liệu ban đầu. Đây là hệ quả tất yếu của việc các tiến trình song song cùng tranh giành quyền cập nhật một trạng thái dùng chung.



Hình 6.5: Lost Update anomaly — hai saga ghi đè kết quả của nhau

Ví dụ trong KBM: một sinh viên nộp hai bài liên tiếp, bài A được cộng 10 điểm và bài B được cộng 5 điểm, và hai bài được xử lý đồng thời. Cả hai luồng xử lý đều tiến hành truy xuất bảng điểm hiện tại và cùng đọc được con số gốc là 100 điểm. Luồng A cập nhật cơ sở dữ liệu thành 110, trong khi luồng B chậm hơn một phần nghìn giây nên đã ghi đè lên thành 105. Kết quả là hệ thống ghi nhận 105 điểm thay vì 115: điểm của lượt nộp A đã mất.

2. **Dirty Read (Đọc dữ liệu rác/chưa hoàn tất)** — Kịch bản rủi ro thứ hai xảy ra khi Saga A đọc và sử dụng những dữ liệu trung gian mà Saga B vừa mới ghi xuống nhưng thực chất lại chưa hoàn thành trọn vẹn (và hoàn toàn có nguy cơ sẽ bị rollback trong tương lai gần).

   Ví dụ về Dirty Read: Nếu saga chấm bài cộng điểm cho sinh viên ngay khi nộp bài, tiến trình đọc của Leaderboard lập tức thấy số điểm này và hiển thị. Tuy nhiên, nếu Judge Service sau đó gặp lỗi và saga kích hoạt bù trừ (trừ điểm lại), Leaderboard đã vô tình hiển thị mức điểm chưa chính thức. KBM tránh được lỗi này bằng cách đặt "Update Score" ở cuối dưới dạng giao dịch Retriable, nghĩa là điểm chỉ được cập nhật sau khi chấm xong hoàn toàn.

3. **Non-repeatable/Fuzzy Read (Đọc dữ liệu không nhất quán)** — Tình huống thứ ba xuất hiện khi một tập dữ liệu bất ngờ thay đổi giá trị ngay giữa hai lần thực hiện truy vấn đọc trong khuôn khổ của cùng một saga duy nhất.

   Trở lại với hệ thống KBM: hãy hình dung saga "chốt kết quả kỳ thi". Ở bước đầu, saga đọc bảng điểm (truy vấn đọc lần một) để xác định top 3 chuẩn bị trao giải, rồi lần lượt gọi các

service khác để khóa kỳ thi và sinh chứng nhận. Đến bước cuối, khi đọc lại bảng điểm (truy vấn đọc lần hai) để ghi thứ hạng chính thức, dữ liệu đã khác đi — vài saga chấm bài nộp sát giờ vừa hoàn tất trong lúc saga này đang chạy. Hai lần đọc *trong cùng một saga* cho hai kết quả khác nhau — đó chính là Fuzzy Read.

ACD thay vì ACID

Richardson trong [2a, Ch.4] chỉ ra: Saga chỉ đảm bảo **ACD** (Atomicity, Consistency, Durability) — *không có Isolation*. Atomicity đạt được nhờ compensating transaction (rollback logic). Consistency bảo toàn bởi quy tắc nghiệp vụ (business rule) trong mỗi local transaction. Durability do mỗi database đảm bảo. Nhưng Isolation phải được xử lý riêng — bằng **các biện pháp đối phó**.

6.4.3. Các biện pháp đối phó (countermeasure)

Richardson đề xuất các biện pháp đối phó [2a, Ch.4], áp dụng cho KBM:

Semantic Lock — Đặt cờ "đang xử lý" trên record, ngăn saga khác đọc/ghi data chưa final:

Listing 6.2: *Semantic Lock — đặt cờ "JUDGING" để chặn thao tác trên dữ liệu chưa chốt*

```
// Chặn nộp đúp ngay ở tầng DB bằng partial unique index
// (tránh lỗi TOCTOU của check-then-insert):
// CREATE UNIQUE INDEX uq_active_submission ON submissions (user_id, question_id)
//   WHERE status IN ('PENDING', 'JUDGING', 'CANCELLING');
try {
    submissionRepository.saveAndFlush(Submission.pending(userId, questionId, sql));
} catch (DataIntegrityViolationException ex) {
    // Ném runtime exception để Spring rollback transaction
    // và trả về HTTP 409 Conflict
    throw new ConflictException("SUBMISSION_IN_PROGRESS",
        "Đang có bài chờ chấm cho câu hỏi này.");
}

// Khi saga bắt đầu — dùng Atomic SQL Update để giữ khóa nghiệp vụ:
// UPDATE submissions SET status = 'JUDGING' WHERE id = ? AND status = 'PENDING'
int rows = submissionRepository.markAsJudging(submissionId);
if (rows == 0) {
    throw new ConflictException("SUBMISSION_ALREADY_LOCKED",
        "Submission đang được xử lý hoặc không còn ở trạng thái PENDING.");
}

// Khi saga hoàn thành — giải phóng khóa
// (UPDATE ... WHERE id = ? AND status = 'JUDGING')
submissionRepository.markAsJudged(submissionId);
```

Lưu ý việc chặn nộp đúp không dùng `existsBy...` ở tầng ứng dụng rồi mới `INSERT`, vì hai yêu cầu đến đồng thời vẫn lọt qua khe hở kiểm tra–ghi (TOCTOU). Ràng buộc `UNIQUE` có điều kiện (`WHERE status IN ('PENDING', 'JUDGING', 'CANCELLING')`) ép cơ sở dữ liệu phân xử nguyên tử và trả về mã `409 Conflict` thay vì `400 Bad Request` (dữ liệu đầu vào hợp lệ nhưng xung đột với trạng thái hiện hành của tài nguyên, xem §3.1).

Commutative Updates — Thiết kế thao tác cập nhật không phụ thuộc thứ tự — giải quyết **lost update**:

Listing 6.3: Commutative Updates — cập nhật theo lượng thay đổi (delta) thay vì giá trị tuyệt đối

```
// SAI: Không commutative: set absolute value — thứ tự quan trọng
score.setValue(150);

// VẪN SAI: delta update nhưng là read-modify-write trên RAM,
// khi flush vẫn sinh "SET value = <giá trị tuyệt đối>" → Lost Update y nguyên.
// Ngoài ra không idempotent: event gửi lại (retry) sẽ cộng 20, 30 điểm.
score.incrementValue(+10);

// ĐÚNG: Commutative + Idempotent (trong cùng @Transactional):
// 1) INSERT INTO processed_events(event_id, processed_at)
//    VALUES (:eventId, now()) ON CONFLICT (event_id) DO NOTHING
if (processedEventRepository.tryMarkProcessed(eventId) == 0) {
    return; // Đã xử lý event này, thoát an toàn mà không làm hỏng transaction
}
// 2) UPDATE scores SET value = value + :delta WHERE user_id = :userId
scoreRepository.incrementScoreAtomically(userId, 10);
```

Chiến thuật Commutative Updates (Cập nhật giao hoán) là một giải pháp hiệu quả cho vấn đề Lost Update. Thay vì ghi đè một giá trị tuyệt đối (ví dụ: gán điểm số = 150), hệ thống áp dụng một phép toán thay đổi tương đối (delta, ví dụ: +10 điểm). Nhờ tính chất giao hoán của phép cộng, dù các sự kiện cập nhật điểm đến không theo thứ tự, kết quả cuối cùng vẫn chính xác. Cơ chế này rất phổ biến trong các hệ thống tính điểm, kế toán hoặc ví điện tử. Tuy nhiên, việc áp dụng cập nhật tương đối khiến hệ thống mất đi idempotency tự nhiên. Nếu một sự kiện (+10 điểm) bị broker phát lại hai lần, dữ liệu sẽ bị cộng dồn sai lệch.

⚠ Lỗ hổng Idempotency với Commutative Updates

Tuy phép cộng có tính giao hoán, nhưng nó *không* idempotent. Vì bước Update Score là một *Retriable transaction* (được retry liên tục nếu lỗi mạng), nếu thao tác cộng điểm nguyên tử (`+10`) bị gọi lại 3 lần mà không kiểm tra trùng, sinh viên sẽ bị cộng lặp thành 30 điểm (Double-counting)! Bắt buộc phải kết hợp với khóa chống trùng lặp (ví dụ: ghi `eventId` vào bảng `processed_events` bằng `INSERT ... ON CONFLICT DO NOTHING` trong cùng transaction hoặc dùng Inbox pattern) để đảm bảo không bị cộng dồn sai lệch.

Ba biện pháp còn lại nhắm vào dirty read, non-repeatable read và những lost update còn sót:

Pessimistic View (Góc nhìn bi quan) — Sắp xếp lại thứ tự các bước trong saga để giảm thiểu nguy cơ dirty read. Các bước không thể đảo ngược hoặc cập nhật tích lũy (như update score) sẽ được đặt *sau* pivot transaction. KBM tuân thủ nguyên tắc này khi chỉ cộng điểm sinh viên sau khi Judge ra phán quyết cuối cùng.

Re-read Value (Đọc lại giá trị) — Đây là hình thức áp dụng optimistic locking: thay vì dùng dữ liệu đã truy xuất từ trước, saga thực hiện đọc lại dữ liệu (re-read) ngay trước khi ghi đè để kiểm tra xem có thay đổi nào xảy ra không.

Listing 6.4: Re-read Value — kiểm tra lại trước khi quyết định

```
// Trước khi update score, không nên sử dụng câu lệnh `if` để kiểm tra
// trên bộ nhớ ứng dụng vì dễ mắc lỗi TOCTOU
// (Time-of-Check to Time-of-Use). Phải đẩy kiểm tra xuống tầng DB:
// UPDATE scores SET value = value + 10
// WHERE user_id = ? AND NOT EXISTS (
```

```
//   SELECT 1 FROM submissions WHERE id = ? AND status = 'CANCELLED'
// )
// Lưu ý: subquery chéo bảng này chạy được vì scores và submissions
// hiện còn chung database (hiện trạng KBM). Sau khi tách database,
// Score Service phải kiểm tra trạng thái qua API/event thay vì SQL.
scoreRepository.incrementIfSubmissionNotCancelled(
    userId, 10, submissionId
);
```

Đẩy khối kiểm tra điều kiện (condition check) trực tiếp xuống lớp cơ sở dữ liệu thay vì xử lý trong bộ nhớ của ứng dụng giúp thu hẹp khe hở Time-of-Check to Time-of-Use (TOCTOU), nhưng chưa đóng hẳn: ở mức READ COMMITTED, subquery không khóa dòng `submissions`. Muốn đóng hẳn, khóa dòng đó bằng `SELECT status FROM submissions WHERE id = ? FOR SHARE` trong cùng transaction rồi mới cập nhật, hoặc chạy ở mức SERIALIZABLE.

Version File (Tệp tin phiên bản) — Để quản lý chặt chẽ những tình huống mà nhiều luồng saga tranh chấp nhau cập nhật cùng một hồ sơ, kỹ thuật cuối cùng được nhắc tới là lưu trữ lại một cuốn sổ cái ghi nhận thứ tự xuất hiện của mọi tác vụ. Từ đó cho phép hệ thống sắp xếp lại (reorder) các thao tác này theo một trật tự mong muốn trước khi áp dụng vào dữ liệu thực.

Giả sử Saga A và Saga B cùng cập nhật điểm số, Version File cho phép hệ thống ghi nhận rồi sắp xếp lại các thao tác về đúng trật tự trước khi áp dụng. Với KBM, hướng đơn giản hơn là *tuần tự hóa* thông điệp điểm số trên cùng một partition (key theo sinh viên) ngay từ đầu vào, khiến xung đột ghi đè đồng thời không còn cơ hội phát sinh — tức là *phòng ngừa* out-of-order từ gốc, thay vì lưu lại rồi reorder như Version File nguyên bản. Hiện `judge-results` dùng key `submissionId` nên KBM chưa có đảm bảo này.

6.4.4. Tổng hợp: anomaly và biện pháp đối phó

Để giải quyết ba anomaly trên, KBM cần các biện pháp đối phó tương ứng (phần lớn nằm trong lộ trình ở §6.6). Với tình trạng **Lost update**, thay vì ghi đè giá trị tuyệt đối, hệ thống cần chuyển sang **Commutative updates** — delta được áp *nguyên tử ngay ở tầng database* (`UPDATE scores SET value = value + ?`), không phải tăng giá trị trên object trong bộ nhớ — để thứ tự ghi nhận điểm số không làm sai lệch kết quả cuối; khi các thao tác không có tính giao hoán, có thể kết hợp **Version file** (hoặc tuần tự hóa thông điệp điểm số theo `userId` trên cùng một Kafka partition) để chống ghi đè lệch thứ tự. Nhằm chặn hiện tượng **Dirty read**, KBM cần **Semantic lock** (bật cờ trạng thái `JUDGING`) kết hợp **Pessimistic view** (chỉ cập nhật điểm sau bước pivot) để ngăn tiến trình khác đọc điểm số chưa chốt. Cuối cùng, trước cả **Non-repeatable read** lẫn xung đột ghi đồng thời, **Re-read value** đóng vai trò chốt chặn kiểm tra lại ngay trong câu lệnh ghi nhằm phát hiện trạng thái đã đổi giữa chừng (chẳng hạn bài nộp vừa chuyển sang `CANCELLED` hoặc bảng điểm vừa cập nhật trước khi chốt kết quả thi).

Các kỹ thuật trên bù đắp cho sự thiếu hụt của tính cô lập (Isolation) trong hệ thống phân tán. Khi gánh nặng duy trì tính toàn vẹn chuyển từ cơ sở dữ liệu sang tầng ứng dụng, thiết kế phải bám sát quy trình nghiệp vụ thực tế (business process); các hành động bù trừ (compensation) vì thế cần phản ánh đúng cách nghiệp vụ xử lý lỗi, thay vì chỉ rollback dữ liệu cơ học.

Compensating Transaction là hành động nghiệp vụ

Compensation trong Saga không phải là `DELETE` hay `ROLLBACK` — đó là **hành động nghiệp vụ có ý nghĩa** phản ánh quy trình thực tế: "đánh dấu đã hủy", "hoàn tiền", "gửi thông báo lỗi tới

user". Điều quan trọng: mỗi compensating transaction phải được **thiết kế cùng lúc** với forward transaction, không phải thêm vào sau. Và tất cả compensation phải idempotent — gọi nhiều lần cho cùng kết quả.

6.5. Eventual Consistency — chấp nhận và quản lý

6.5.1. Định lý CAP và mô hình BASE

Quyết định từ bỏ 2PC để chuyển sang Saga thực chất là một sự đánh đổi có chủ đích liên quan đến định lý CAP. CAP buộc hệ thống phân tán phải chọn giữa Consistency (Nhất quán) và Availability (Sẵn sàng) *khi xảy ra network partition*. Luồng nộp bài của KBM ưu tiên phía **Availability** — vẫn tiếp nhận bài nộp khi một dịch vụ phía sau gián đoạn — và chấp nhận nới lỏng **C** (nhất quán tức thời). Bản thân Saga không phải là "lựa chọn AP": nó là kỹ thuật điều phối giúp hệ thống hội tụ về trạng thái đúng (eventual consistency) sau khi đã chấp nhận đánh đổi đó; từng service vẫn có thể từ chối ghi khi cần bảo toàn bất biến nghiệp vụ (phân tích kỹ hơn ở Ch.7).

Trong thiết kế từ bỏ 2PC như của KBM, các giao dịch *xuyên dịch vụ* không còn giữ được trọn vẹn bảo đảm ACID mà dịch chuyển sang mô hình **BASE** [7, Ch.7] — lưu ý quan trọng: mỗi local transaction bên trong từng service *vẫn là ACID*. Hệ thống BASE hy sinh tính cô lập (Isolation) toàn cục để đổi lấy tính sẵn sàng (Availability) và tính nhất quán cuối cùng (Eventual Consistency), trong khi Saga tạo ra tính nguyên tử ở cấp nghiệp vụ: hoặc toàn bộ chuỗi hoàn tất, hoặc các bước đã thực hiện được bù trừ. Đây không phải rollback theo nghĩa ACID, vì bên khác có thể đã kịp quan sát trạng thái trung gian. Sự chuyển dịch này thể hiện rõ ràng qua từng đặc tính trong thiết kế của KBM:

- **Basically Available (Sẵn sàng cơ bản)**: Hệ thống ưu tiên sự khả dụng cao: luôn trong trạng thái sẵn sàng tiếp nhận các bài nộp mới từ sinh viên, bất chấp việc một vài dịch vụ phía sau có thể đang bị gián đoạn.
- **Soft state (Trạng thái mềm)**: Dữ liệu điểm số có thể tạm thời chưa được cập nhật đầy đủ và vẫn đang thay đổi trong hệ thống, hệ thống không khóa cứng các bản ghi.
- **Eventual consistency (Nhất quán cuối cùng)**: Chấp nhận việc bảng xếp hạng (leaderboard) có độ trễ tạm thời, nhưng cam kết rằng *cuối cùng* mọi dữ liệu sẽ đồng bộ và chính xác.

Lưu ý về **tính bền vững (Durability — chữ D của ACID)**: việc BASE không có chữ `D` không có nghĩa là hệ thống từ bỏ độ bền dữ liệu. Cả hai mô hình đều giữ đặc tính này ở mức local commit — dữ liệu một khi đã commit xuống cơ sở dữ liệu cục bộ vẫn sống sót qua sự cố theo đúng cam kết của tầng lưu trữ; còn retention, replication và backup là những thiết kế vận hành riêng, không tự nhiên mà có.

6.5.2. Consistency window trong KBM

Khi KBM chọn Saga thay cho giao dịch phân tán, hệ thống phải chấp nhận một hệ quả trực tiếp: dữ liệu giữa các dịch vụ có độ trễ. Trong một giao dịch nguyên khối, mọi bảng dữ liệu được đồng bộ tại đúng thời điểm commit; với saga, giữa các local transaction luôn tồn tại khoảng quá độ mà thông tin ở các dịch vụ chưa khớp nhau. Nhiệm vụ của thiết kế không phải là xóa bỏ khoảng trễ đó, mà là kiểm soát nó ở mức người dùng không bị ảnh hưởng.

Consistency Window

Rocha trong [5, Ch.5] định nghĩa **consistency window** (cửa sổ nhất quán): khoảng thời gian từ lúc một sự kiện xảy ra cho đến khi tất cả các dịch vụ liên quan phản ánh trạng thái mới đó. Mục tiêu của người thiết kế không phải là triệt tiêu khoảng thời gian này, mà là giữ độ rộng của consistency window ở mức **đủ nhỏ** để người dùng cuối không nhận ra sự chậm trễ.

Trong quy trình của KBM: sau khi sinh viên nhấn nút nộp bài, consistency window mở ra và kéo dài cho tới khi có kết quả. Độ rộng của cửa sổ này phụ thuộc vào thời gian thực thi mã nguồn và tải của Judge Service. Trong khoảng thời gian chờ đó, thiết kế đề xuất cho KBM dùng các cơ chế sau (semantic lock thuộc Giai đoạn 1 ở §6.6):

- `submission.status = JUDGING` (semantic lock)
- Bảng xếp hạng (Leaderboard) tạm thời hiển thị điểm số cũ chưa được cập nhật
- Giao diện người dùng (UI) hiển thị trạng thái "Đang chấm..." nhằm cung cấp phản hồi trực quan để người dùng an tâm chờ đợi

6.5.3. Chiến thuật cho giao diện và nghiệp vụ

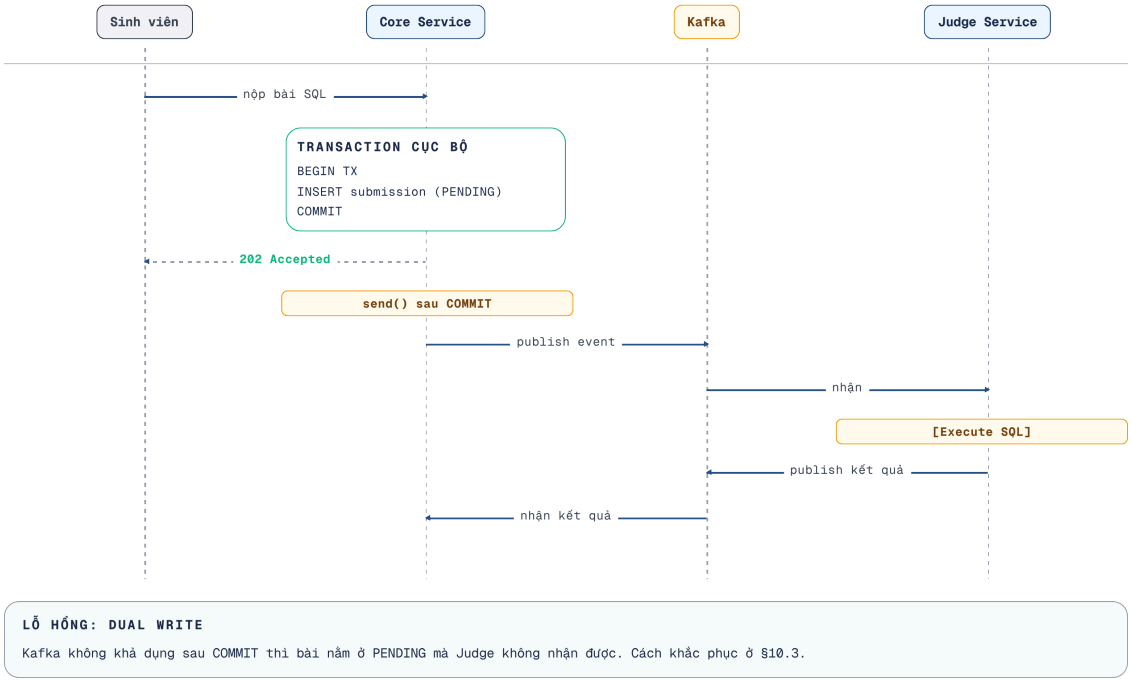
Để lấp khoảng trống của "consistency window", frontend thường kết hợp bốn chiến thuật UI. **Optimistic UI** hiển thị ngay "Bài đã gửi thành công" sau thao tác, dù server phía sau vẫn đang chấm. **Progress indicator** cập nhật các trạng thái trung gian ("Đang chấm...", "Đang so sánh...") để người dùng biết hệ thống không bị treo. Khi xử lý hoàn tất, **Push notification** (qua WebSocket) truyền kết quả cuối cùng lên màn hình. Nếu Judge service gặp lỗi và saga phải bù trừ, **Compensation notification** hiển thị thông báo "Bài không chấm được, vui lòng thử lại" — đóng vòng phản hồi một cách an toàn. Về phía nghiệp vụ, cần chốt với người dùng khoảng chờ chấp nhận được (ví dụ kết quả chấm trong 45 giây ở kịch bản chất lượng §1.5) và cách xử lý khi quá hạn: chấm lại tự động hay yêu cầu nộp lại.

6.6. Case study: KBM

Quá trình sinh viên nộp bài thực chất là một chuỗi hành động trải dài trên nhiều dịch vụ. Việc phân tích luồng xử lý này giúp nhận ra các chỗ thiếu trong thiết kế hiện tại.

6.6.1. Hiện trạng: Implicit Saga (Choreography)

KBM hiện có một implicit saga — choreography qua sự kiện Kafka, nhưng *không* được định nghĩa rõ ràng như saga:



Hình 6.6: *Implicit Saga trong KBM — choreography qua sự kiện Kafka*

Richardson trong [2a, Ch.4] mô tả hai cách điều phối saga đều hợp lệ: **choreography** (các service tự phản ứng theo event) và **orchestration** (bộ điều phối tường minh quản lý state machine kèm compensation). Dù chọn cách nào, một saga đúng nghĩa vẫn cần trạng thái, timeout và compensation *tường minh*. Khi xem xét luồng nộp bài (Submit flow) hiện tại của KBM, thiết kế choreography ngầm định đang thiếu chính các cơ chế an toàn thiết yếu đó (safety mechanism). Việc thiếu saga tường minh khiến chuỗi tiến trình khó kiểm soát. Thiếu timeout và compensation, một lỗi mạng hay Judge dừng đột ngột có thể để bài nộp treo ở `PENDING` mà không ai xử lý — đúng như rủi ro kiến trúc phân tích dưới đây.

Saga không có timeout tại KBM

Trong thiết kế Choreography hiện tại của KBM cho luồng chấm bài, `kbm-core` lưu trạng thái `PENDING` rồi đẩy thông điệp nộp bài vào topic `submissions`. Tuy nhiên, nếu tiến trình gửi thông điệp gặp sự cố hoặc `kbm-judge-sql` dừng đột ngột (crash), sinh viên sẽ thấy bài nộp rơi vào trạng thái chờ đợi (loading) vô thời hạn do không có dịch vụ nào phản hồi và cập nhật trạng thái.

Cách khắc phục được đề xuất là mẫu **Timeout & Compensation** (Giai đoạn 1 ở dưới). Tuy nhiên, nếu chỉ sử dụng một tác vụ theo lịch trình (`@Scheduled` job) để chuyển đổi trạng thái thành `TIMEOUT` một cách máy móc khi quá hạn 5 phút, hệ thống sẽ gặp race condition: Judge Service có thể hoàn thành việc chấm đúng 1 giây sau đó và ghi đè kết quả. Để giải quyết, Orchestrator (Core Service) phải gửi một lệnh "Cancellation/Probe Command" tới Judge Service. Chỉ khi nhận được phản hồi xác nhận hủy (hoặc xác nhận chưa từng chạy) từ Judge, Core Service mới an toàn đổi trạng thái thành `TIMEOUT` và thực hiện các bước Compensation (đánh dấu bài nộp để sinh viên nộp lại). Đây là minh chứng rõ nét: trong hệ thống phân tán, *luôn phải thiết kế cho trường hợp thành phần khác không bao giờ trả lời.*

Vùng xám giữa Choreography và Orchestration. Việc Core Service chủ động gửi "Cancellation Command" (lệnh) tới Judge và lắng nghe "ACK" đã khiến nó hành xử giống như một **Lightweight Orchestrator** thay vì Choreography thuần túy (vốn chỉ phát event trung lập). Sự chuyển dịch từ Event-driven sang Command-driven này là rất phổ biến để giải quyết các luồng Timeout hoặc Compensation phức tạp.

6.6.2. Phân tích các vấn đề và lộ trình cải tiến

Đối chiếu với các tiêu chuẩn thiết kế phân tán (Giải pháp khuyến nghị theo Richardson [2a, Ch.4]), luồng Submit hiện tại của KBM bộc lộ bốn lỗ hổng kiến trúc:

- **Implicit saga (Saga ngầm định)**: Luồng xử lý nằm rải rác trong mã nguồn, không có định nghĩa state machine tập trung — lập trình viên khó bao quát toàn bộ quy trình.
- **Thiếu compensation và timeout**: Nếu Judge Service treo hoặc không phản hồi do nghẽn mạng, trạng thái PENDING của bài nộp bị đình trệ vô thời hạn.
- **Không có semantic lock**: Sinh viên có thể nộp bài mới khi hệ thống vẫn đang xử lý bài cũ, vô tình hoặc cố ý.
- **Thiếu cơ chế theo dõi Saga**: Hệ thống không theo dõi được tiến độ hiện tại của chuỗi (PENDING → JUDGING → CANCELLING → JUDGED / ERROR / TIMEOUT).

Để khắc phục, lộ trình nâng cấp kiến trúc được chia làm ba giai đoạn. **Giai đoạn 1 — Semantic Lock + Timeout** mang tính cấp bách nhất, tập trung vào việc áp dụng khóa nghiệp vụ và cơ chế bù trừ khi quá hạn nhằm tránh race condition giữa timeout và kết quả chấm.

Listing 6.5: Semantic Lock + Probe/Timeout compensation để tránh race condition

```
// Scheduled job kiểm tra timeout
// (2 cấp: Probe mềm sau 5 phút → Hard Timeout sau 10 phút)
@Scheduled(fixedRate = 60000)
public void checkSubmissionTimeouts() {
    // Cấp 1 (quá 5 phút ở PENDING hoặc JUDGING):
    // chuyển sang CANCELLING rồi gửi lệnh hủy/thăm dò
    List<Submission> stuck =
            submissionRepository.findStuckInPendingOrJudging(cutoff5m);
    for (Submission s : stuck) {
        // UPDATE submissions SET status = 'CANCELLING'
        // WHERE id = :id AND status IN ('PENDING', 'JUDGING')
        if (submissionRepository.markAsCancelling(s.getId()) > 0) {
            // Thực tế nên ghi qua Outbox (§10.3)
            judgeCommandProducer.sendCancelCommand(s.getId(), s.getJudgeRunId());
        }
    }

    // Cấp 2 — Hard Timeout
    // (quá 10 phút vẫn kẹt ở CANCELLING vì Judge chết, không có ACK):
    // dùng chung luồng bù trừ để sinh viên vẫn được thông báo
    // và thử lại / đưa vào DLQ
    List<String> deadIds = submissionRepository.findStuckInCancelling(cutoff10m);
    deadIds.forEach(compensator::compensateTimedOutSubmission);
}

// Listener nhận ACK hủy từ kbm-judge-sql
// (trường hợp Judge còn sống và xác nhận đã dừng)
@KafkaListener(topics = "judge-cancel-ack")
public void handleJudgeCancelAck(String submissionId) {
```

```
    compensator.compensateTimedOutSubmission(submissionId);
}

@Transactional
public void compensateTimedOutSubmission(String submissionId) {
    // Nằm trong bean riêng (SubmissionCompensator)
    // được inject vào job và listener ở trên:
    // gọi qua proxy của Spring thì @Transactional mới có hiệu lực;
    // gọi nội bộ (this::...) sẽ bỏ qua giao dịch.
    // Atomic transition: UPDATE submissions SET status = 'TIMEOUT'
    // WHERE id = :id AND status = 'CANCELLING'
    // (chống chạy đúp giữa ACK đến muộn và Hard Timeout)
    if (submissionRepository.markAsTimeoutFromCancelling(submissionId) == 0) {
        return;
    }
    Submission s = submissionRepository.findById(submissionId).orElseThrow();
    // Ba tác dụng phụ ghi vào outbox trong cùng transaction (§10.3):
    // transaction rollback thì không thông báo hay message nào lọt ra ngoài
    if (s.getRetryCount() < MAX_RETRIES) {
        // Lưu bản thử lại (status = PENDING, retryCount + 1, judgeRunId mới)
        Submission retry = submissionRepository.save(s.cloneForRetry());
        outbox.enqueueNotification(s.getUserId(),
                "Judge quá hạn — hệ thống đang chấm lại tự động");
        outbox.enqueueSubmission(retry);
    } else {
        outbox.enqueueNotification(s.getUserId(),
                "Không thể chấm bài lúc này — vui lòng nộp lại sau");
        outbox.enqueueDeadLetter(s); // Vượt ngưỡng retry → DLQ để đối soát
    }
}
```

Fencing cho kết quả đến muộn

Hard timeout là biện pháp cuối cùng khi Judge không phản hồi ACK, nhưng nó mở ra một rủi ro đối xứng: kết quả *hợp lệ* vẫn có thể đến sau khi hệ thống đã ép TIMEOUT (Judge chỉ nghẽn mạng chứ không chết), hoặc kết quả của lần chạy cũ đến sau khi đã có bản retry mới.

Vì vậy mọi handler ghi trạng thái — kể cả handler nhận kết quả chấm — đều phải là transition có điều kiện trên state machine (`UPDATE ... WHERE status = ...`). Mỗi lần chấm cũng nên mang một `judgeRunId` riêng: chỉ kết quả khớp lần chạy đang hiệu lực mới được chấp nhận, còn kết quả "mồ côi" bị bỏ qua và ghi log để đối soát.

Giai đoạn 2 — Định nghĩa Saga tường minh (đòi hỏi mức độ nỗ lực trung bình):

- Định nghĩa `SubmitSaga` class với state machine: PENDING → JUDGING → CANCELLING → JUDGED / ERROR / TIMEOUT
- Mỗi transition có điều kiện và xử lý lỗi rõ ràng; bước compensatable có compensation, còn các bước sau pivot có cơ chế retry
- Log saga state changes cho auditing và debugging

Giai đoạn 3 — Saga Orchestrator (đòi hỏi nhiều nguồn lực, khi cần mở rộng quy mô):

- Cân nhắc khi luồng nộp bài mở rộng (thêm plagiarism check, grading rubric)
- Hiện tại choreography vẫn đủ — luồng xử lý 4 bước, không phân nhánh
- Ngưỡng: khi luồng nghiệp vụ > 4 bước hoặc có phân nhánh phức tạp → chuyển sang orchestration

6.6.3. Workflow học vụ cũng là saga

KBM không chỉ có luồng nộp bài. Các luồng như duyệt bài tập, phê duyệt đề tài/đồ án, công bố điểm hoặc cập nhật CLO cũng có bản chất saga: nhiều bước, nhiều người tham gia, có trạng thái trung gian và có hành động bù trừ khi bị từ chối hoặc quá hạn.

Nhiều quy trình học vụ ánh xạ được vào ba loại bước: Compensatable, Pivot và Retriable:

- **Quy trình phê duyệt bài tập (Assignment approval)**: Giai đoạn đầu tiên (Compensatable) là khi giảng viên tạo bản nháp (Draft assignment) — lúc này mọi thông tin vẫn có thể hủy hoặc sửa dễ dàng. Điểm Pivot chính là khoảnh khắc giảng viên hoặc ban quản lý quyết định bấm nút phê duyệt (Approve). Một khi đã phê duyệt, hệ thống tiến vào chuỗi Retriable bắt buộc phải thành công: xuất bản bài tập ra khóa học và tự động gửi thông báo đến toàn bộ sinh viên.
- **Quy trình xét duyệt đồ án/khóa luận (Thesis/project review)**: Sinh viên khởi tạo đề xuất đề tài (Submit proposal), một hành động Compensatable vì hoàn toàn có thể rút lại. Nút thắt Pivot rơi vào quyết định chính thức của hội đồng chuyên môn (Committee decision). Sau quyết định này, hệ thống sẽ thực thi các tác vụ Retriable như cập nhật trạng thái phê duyệt cuối cùng và thông báo kết quả cho cả sinh viên lẫn giảng viên hướng dẫn.
- **Quy trình công bố điểm (Grade publication)**: Hệ thống bắt đầu bằng việc tính toán điểm số dự kiến (Compute provisional grade) — một bước Compensatable vì có thể bị tính lại nhiều lần. Pivot transaction xuất hiện khi giảng viên xác nhận điểm lần cuối (Lecturer confirms). Ngay sau đó, quá trình Retriable sẽ phụ trách việc công bố bảng điểm chính thức và đồng bộ dữ liệu vào mô hình đọc (read model) để phục vụ cho công tác báo cáo thống kê.

Ở cả ba workflow, bước "thông báo" nằm trong chuỗi Retriable phải được hiểu theo đúng ranh giới đã phân tích ở mục T4: thứ bắt buộc thành công là việc *ghi bền vững lệnh thông báo* còn việc gửi email/push đến người nhận chạy ngoài saga theo kiểu best-effort.

Các workflow này thường nên bắt đầu bằng **explicit state machine** hơn là orchestrator riêng. Khi số bước và nhánh tăng, orchestration mới đáng cân nhắc.

⚠ Sai lầm thường gặp khi triển khai Saga

- **Ép nhất quán mạnh bằng 2PC**: Thói quen ACID từ kiến trúc nguyên khối khiến nhiều kỹ sư cố giữ strong consistency xuyên các dịch vụ, đổi lại là khóa tài nguyên chặn luồng và giảm tính sẵn sàng. *Phòng tránh*: Chấp nhận eventual consistency ở nơi nghiệp vụ cho phép và dùng saga.
- **Chỉ thiết kế cho happy path**: Quên hành động bù trừ (compensation) cho các bước có thể hoàn tác, dẫn đến hệ thống kẹt ở trạng thái trung gian với dữ liệu mâu thuẫn. *Phòng tránh*: Mỗi bước compensatable đều phải có compensation tường minh và đã được kiểm thử.
- **Không kiểm soát timeout**: Saga bắt đầu nhưng không ai theo dõi thời gian thực thi sẽ treo vĩnh viễn khi mạng chập chờn hoặc dịch vụ dừng đột ngột. *Phòng tránh*: Dùng tác vụ chạy ngầm quét saga quá hạn và tự kích hoạt bù trừ.
- **Bỏ qua semantic lock**: Không khóa dữ liệu đang xử lý cho phép người dùng thao tác đúp (sinh viên nộp lại bài khi hệ thống đang chấm), sinh ra race condition và dữ liệu rác. *Phòng tránh*: Áp dụng semantic lock ngay từ bước đầu của saga (§6.4.3).
- **Compensation không idempotent**: Khi bước bù trừ được thực thi lại, hệ thống bù trừ hai lần và làm sai lệch dữ liệu. *Phòng tránh*: Thiết kế mọi hành động bù trừ bảo đảm idempotency (§6.4.1).

- **Lạm dụng choreography cho saga nhiều nhánh**: Tạo ra ma trận sự kiện khó theo dõi và dễ phụ thuộc vòng. *Phòng tránh*: Khi quy trình vượt quá bốn bước hoặc có phân nhánh phức tạp, chuyển sang orchestration (§6.3.3).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Saga Pattern (Orchestration)**: saga-orchestration.html

6.7. Tổng kết

Giao dịch phân tán (distributed transaction) là một trong những bài toán khó nhất khi chuyển từ monolith sang microservices. Two-Phase Commit — giải pháp truyền thống — không phù hợp vì blocking, single point of failure, và giảm availability.

Saga pattern giải quyết bằng cách thay một ACID transaction bằng chuỗi local transaction + compensating transaction. Ba loại transaction (compensatable, pivot, retriable) tạo cấu trúc rõ ràng cho mỗi saga. Choreography (phi tập trung) phù hợp cho các saga đơn giản, orchestration (tập trung) cho các saga phức tạp.

Thiếu isolation là thách thức lớn nhất. Semantic lock, commutative updates, pessimistic view, re-read value và version file là năm biện pháp đối phó giúp giảm thiểu các anomaly — tất cả đều áp dụng được cho KBM.

Eventual consistency là trade-off có chủ đích cho tính sẵn sàng và khả năng mở rộng. Trong KBM, consistency window ngắn là chấp nhận được — sinh viên quen với việc chờ kết quả chấm bài. Quản lý consistency window bằng semantic lock và real-time notification là đủ cho bài toán hiện tại.

Phân tích KBM cho thấy hệ thống đang dùng implicit saga (choreography) và cần làm rõ compensation, semantic lock, timeout handling. Bài nộp hoặc các luồng quy trình học vụ có thể bị đình trệ ở trạng thái trung gian — đây là nợ kỹ thuật nghiêm trọng cần xử lý ưu tiên (Giai đoạn 1).

Ở Chương 7, chúng ta sẽ giải quyết bài toán **quản lý dữ liệu** — database-per-service, CQRS, Event Sourcing — và phân tích sâu hơn vấn đề cơ sở dữ liệu dùng chung trong KBM.

6.8. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.4: Managing Transactions with Sagas
2. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.4: Sagas (Choreography, Orchestration); Ch.5: Eventual Consistency
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.7: Transactions; Ch.9: Consistency and Consensus

Nguồn trực tuyến:

- Gregor Hohpe, "Starbucks Does Not Use Two-Phase Commit" (2004) — enterpriseintegrationpatterns.com
- Caitie McCaffrey, "Applying the Saga Pattern" (GOTO Chicago 2015) — youtube.com

- Eventuate Tram Sagas framework — eventuate.io

Bài tập: 6-0, 6-1, 6-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.